

DOCUMENTO DE DECISIONES DE DISEÑO

ENTIDADES ELEGIDAS Y JUSTIFICACION

Se identificaron 11 entidades para PostgreSQL y 2 colecciones para MongoDB. La selección se basó en el análisis de los requerimientos funcionales RF-01 al RF-12 y las reglas de negocio RN-01 al RN-08.

Entidades PostgreSQL

Entidad	Justificación	RF/RN relacionado
usuarios	Personal que opera el sistema. Separada de médicos porque un médico tiene datos clínicos y un recepcionista no.	RF-12
especialidades	Catálogo de especialidades. Entidad propia para evitar repetir el nombre en cada médico.	RF-01
médicos	Datos profesionales del médico vinculados a una especialidad.	RF-02
horarios_médico	Bloques semanales de atención. Tabla separada porque un médico puede tener múltiples horarios.	RF-02, RN-02
pacientes	Datos personales y de contacto. Entidad central del sistema.	RF-03
citas	Núcleo del sistema. Un médico, paciente y fecha/hora con su ciclo de vida.	RF-04, RF-06, RN-01..03
servicios	Catálogo de consultas, procedimientos y exámenes con precio.	RF-09
facturas	Encabezado de factura. Una por cita atendida.	RF-10, RN-04..06
factura_detalle	Líneas de servicios con snapshot de precio para proteger historial.	RF-10
pagos	Pagos parciales o totales. Una factura puede tener múltiples pagos.	RF-11
auditoria_log	Log de operaciones críticas con JSONB para payload variable.	RF-12

Colecciones MongoDB

Colección	Justificación
historiales_clínicos	Información clínica variable por especialidad. Schema flexible = MongoDB.
auditoria_eventos	Log de alto volumen con estructura variable según el tipo de operación.

DIVISION POSTGRESQL vs MONGODB

Por que PostgreSQL para datos financieros

Las facturas y pagos requieren ACID estricto. Registrar un pago es una operacion atomica de 5 pasos: validar factura, validar monto, insertar pago, actualizar estado, registrar auditoria. Si falla cualquier paso, ningun cambio debe persistir. PostgreSQL garantiza esto con BEGIN/EXCEPTION/ROLLBACK en los stored procedures `sp_registrar_pago` y `sp_cancelar_cita`.

Por que MongoDB para historiales clinicos

La informacion clinica varia sustancialmente entre especialidades. Un cardiologo registra electrocardiograma y fraccion de eyeccion. Un pediatra registra vacunas, percentil de talla y peso. Un dermatologo registra tipo de lesion, localizacion y fototipo.

Modelar esto en PostgreSQL requeriria decenas de columnas con valores NULL o una tabla EAV (Entity-Attribute-Value) dificil de consultar y mantener. MongoDB permite el campo `datos_especialidad` como objeto libre (Schema.Types.Mixed) que se adapta a cada especialidad sin modificar el schema.

Por que MongoDB para auditoria de alto volumen

El log de auditoria puede crecer a cientos de entradas diarias. Cada entrada tiene un payload variable segun la operacion un pago tiene datos distintos a una cancelacion de cita. MongoDB maneja bien este patron de escritura intensiva con estructura variable, y sus pipelines de aggregation permiten analisis flexibles.

REGLAS DE NEGOCIO: SCHEMA vs STORED PROCEDURE

Implementadas a nivel de schema (DDL)

Estas reglas se implementan como constraints porque son simples, inmediatas y no requieren logica adicional:

Regla	Mecanismo	Ubicacion
RN-07: cancelacion requiere motivo	CHECK(estado <> 'cancelada' OR motivo IS NOT NULL)	tabla citas
Integridad referencial	FOREIGN KEY en todas las relaciones	todas las tablas
Unicidad medico+horario	UNIQUE(medico_id, fecha_hora)	tabla citas
Precio positivo	CHECK(precio >= 0)	servicios, facturas
Fecha nacimiento no futura	CHECK(fecha_nacimiento <= CURRENT_DATE)	tabla pacientes
Estado de cita valido	CHECK(estado IN (lista))	tabla citas

Implementadas a nivel de stored procedure

Estas reglas involucran multiples tablas o calculos que requieren logica procesal:

Regla	SP	Razon
RN-01: sin solapamiento de citas	sp_agendar_cita	Requiere FOR UPDATE para manejar concurrencia entre transacciones
RN-02: cita dentro de horario	sp_agendar_cita	Requiere consultar tabla horarios_medico
RN-03: 1 cita activa/medico/dia	sp_agendar_cita	Requiere COUNT sobre tabla citas con filtros multiples
RN-04: pagos no exceden total	sp_registrar_pago	Requiere SUM de pagos existentes antes de insertar
RN-05: no pagar factura anulada	sp_registrar_pago	Requiere verificar estado actual de la factura
RN-06: estado factura automatico	sp_registrar_pago	Usa fn_calcular_estado_factura() para determinar el estado

DECISIONES DE MONGODB

Embebido vs Referencias

Los arrays diagnosticos, medicamentos y exámenes se guardan EMBEBIDOS dentro del documento del historial clinico. Justificacion: siempre se consultan juntos con el historial — nunca se necesita un diagnostico sin su historial. Embeber evita \$lookup entre colecciones y mejora el rendimiento de lectura.

Denormalizacion

Los campos especialidad, nombre_paciente y nombre_medico se guardan denormalizados en el historial. Justificacion: los pipelines de aggregation de MongoDB no pueden hacer JOIN con PostgreSQL. Denormalizar estos campos permite queries eficientes en los pipelines RC-07, RC-08 y RC-09 sin salir de MongoDB.

Campo datos_especialidad

Se usa Schema.Types.Mixed para permitir estructura libre por especialidad. Este es exactamente el caso de uso para el que MongoDB fue diseñado: datos con estructura variable que no se conoce completamente al diseño inicial del schema.

NORMALIZACION

El modelo relacional esta normalizado hasta la 3a Forma Normal (3FN). Todas las dependencias funcionales son directas a la clave primaria de cada tabla, sin dependencias transitivas.

La unica desnormalizacion es factura_detalle.subtotal como columna generada (GENERATED ALWAYS AS cantidad * precio_unitario STORED). Esta desnormalizacion esta justificada porque:

1. El subtotal depende directamente de atributos de la misma fila, por lo que técnicamente no viola la 3FN.
2. Evita recalcular en cada consulta mejorando el rendimiento.
3. El valor es siempre consistente porque lo calcula PostgreSQL automáticamente.